

Marcar como hecha

11.1. Características y ventajas de un servlet. Creación de servlets HTTP

Los servlets son bloques de construcción básicos para construir interfaces de base Web para aplicaciones. La tecnología de servlets proporciona un modelo de programación común que es también la base de las Páginas JavaServer.

El objetivo es presentar el API Java Servlet. Este API incluye dos paquetes:

javax.servlet

javax.servlet.http.

El API Java Servlet se puede analizar en cuatro pasos:

1. Implementación de servlets
2. Solicitudes y respuestas
3. Sesión, contexto y colaboración de servlet
4. Filtrado de solicitudes y respuestas

En principio, veremos los dos primeros y examinaremos los siguientes puntos:

Clases e interfaces para la implementación de servlets, incluido las excepciones de servlet

- Configuración de servlets
- El ciclo de vida de los servlets
- Solicitudes y respuestas
- El modelo de programación del servlet

No se pretende una referencia exhaustiva para los paquetes javax.servlet y javax.servlet.http. Si, en cambio, analizar estos paquetes y los conceptos que los respaldan, concentrándonos en cómo utilizar el API durante el desarrollo de las aplicaciones.

La documentación del API Servlet debe tenerse cerca.

<https://docs.oracle.com/javaee/7/api/index.html>

Análisis del API Java Servlet

Como ya hemos visto, el API Servlet está especificado en dos paquetes de extensión Java: javax.servlet y javax.servlet.http. **Las clases e interfaces contenidas en el paquete javax. servlet son independientes del protocolo, mientras que el segundo paquete, javax.servlet.http, contiene clases e interfaces que son específicas de HTTP.** Observe que algunas de las clases/interfaces en javax.servlet.http amplían las especificadas en el paquete javax.servlet.

¿Cuál es el propósito de todas estas interfaces y clases? Consideremos las siguientes cuestiones:

- ¿Cómo gestiona el contenedor Web el ciclo de vida de los servlets? ¿Cómo pueden los servlets participar en este ciclo de vida?
- ¿Cómo implementamos la lógica de aplicación? La lógica de aplicación en servlets es la lógica de programación para responder a las solicitudes HTTP. Dependiendo de la arquitectura y complejidad de las aplicaciones, ésta podría ser acceder a bases de datos, invocar los servicios de diversos EJB u otros componentes de capa media, o utilizar JMS para enviar mensajes a otras aplicaciones de empresa, etc.
- ¿Cómo leemos solicitudes HTTP y cómo generamos respuestas HTTP?
- ¿Cómo tratamos las excepciones en los servlet?
- ¿Cómo pueden los servlets interactuar con su entorno?

La siguiente lista ofrece un análisis del API Java Servlet, que será examinado con detalle en breve. En esta lista, varias interfaces/clases están agrupadas según su propósito y uso (que estudiaremos en un momento).

Implementación de servlet

- javax.servlet.Servlet
- javax.servlet.SingleThreadModel
- javax.servlet.GenericServlet javax.servlet.http.HttpServlet

Configuración de servlet

- javax.servlet.ServletConfig

Excepciones servlet

- javax.servlet.ServletException
- javax.servlet.UnavailableException

Solicitudes y respuestas

- javax.servlet.http.HttpServletRequestWrapper
- javax.Servlet.http.HttpServletResponse
- javax.servlet.http.HttpServletResponseWrapper
- javax.servlet.ServletInputStream
- javax.servlet.ServletOutputStream
- javax.servlet.ServletRequest
- javax.servlet.ServletRequestWrapper
- javax.servlet.ServletResponse
- javax.servlet.ServletResponseWrapper

Localización de sesión

- javax.servlet.http.HttpSession
- javax.servlet.http.HttpSessionActivationListener
- javax.servlet.http.HttpSessionAttributeListener
- javax.servlet.http.HttpSessionBindingListener
- javax.servlet.http.HttpSessionBindingEvent
- javax.servlet.http.HttpSessionEvent
- javax.servlet.http.HttpSessionListener

Contexto de servlet

- javax.servlet.ServletContext
- javax.servlet.ServletContextAttributeEvent
- javax.servlet.ServletContextAttributeListener
- javax.servlet.ServletContextEvent
- javax.servlet.ServletContextListener

Colaboración de servlet Filtrado

- javax.servlet.Filter
- javax.servlet.FilterChain
- javax.servlet.FilterConfig

Varios

- javax.servlet.RequestDispatcher
- javax.servlet.http.Cookie
- javax.servlet.http.HttpUtils

Los servidores Web con las implementaciones de Java Servlet, como Tomcat, WebLogic, Orion, iPlanet, JRun, etc., utilizan la mayoría de las interfaces y clases que aparecen en la tabla anterior. De hecho, aparte de las diversas interfaces escuchadoras de la tabla, sólo queda una clase abstracta (javax.servlet.http.HttpServlet) y una interfaz (javax.servlet. Filter) para implementar cuando construya aplicaciones Web. El contenedor (ver arquitectura JEE) crea el resto de los objetos de periodo de ejecución que son parte de su aplicación. ¿Por qué ocurre así?

?

Un contenedor Web es un marco de aplicación con un entorno de periodo de aplicación. En el marco, los objetos de aplicaciones son creados e invocados por el periodo de ejecución. Mientras invoca los métodos sobre los objetos de aplicación, el contenedor tiene que construir los objetos para

los argumentos.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
```

Del mismo modo, el contenedor crea todos los demás objetos (como ServletConfig, ServletContext, HttpSession, etc.) a los que un servlet puede acceder, directa o indirectamente.

La tabla anterior clasifica las clases/interfaces de los paquetes javax . servlet y javax.servlet.http, basándose en su función en el desarrollo de aplicaciones Web. Por lo tanto, examinemos ahora esta clasificación más detenidamente:

-- Implementación de servlet Las clases/interfaces de esta categoría están destinadas a implementar servlets. Para desarrollar nuestros propios servlets, implementaríamos uno o más métodos de estas clases/interfaces. Cuando desplegamos servlets, el contenedor Web invoca estos métodos para controlar su ciclo de vida y para ejecutar la lógica de aplicación.

--Configuración de servlet La interfaz ServletConfig pertenece a esta categoría. El API Servlet ofrece varios medios de acceder al objeto ServletConfig asociado a un servlet. Este objeto proporciona acceso a ciertos parámetros de inicialización que pueden ser configurados al desplegar un servlet.

--Excepciones servlet El API Java Servlet especifica dos excepciones: ServletException y UnavailableException. Habitualmente, los servlets lanzan estas excepciones, que el contenedor captura luego para ejecutar un apropiado manejo de errores.

--Solicitudes y respuestas Hay cuatro interfaces y dos clases abstractas en esta categoría. Estos objetos proporcionan métodos para acceder a la conexión cliente y las corrientes subyacentes de entrada y salida asociadas a ésta. Utilizando estos objetos, podemos leer datos de la entrada y escribir datos de vuelta al cliente.

Además de las anteriores, las nuevas especificación Servlet introducen clases envoltorio para las clases ServletRequest, ServletResponse, http.HttpServletRequest y http.HttpServletResponse (como hemos visto en la tabla en los nombres de clases terminados en "Wrapper"). Aunque todas estas clases son abstractas, no contienen en realidad ninguna funcionalidad.

--Localización de sesión La localización de sesión es una de las partes esenciales del desarrollo de aplicaciones Web. Como veremos en el capítulo 7, que es el protocolo fundamental para las aplicaciones Web, no tiene estado. Como resultado, las aplicaciones Web no pueden reconocer múltiples solicitudes del mismo cliente HTTP originándose del mismo lugar. La noción de sesión proporciona esta abstracción. En términos sencillos, una sesión le permite agrupar solicitudes en un grupo reunido. Además, la gestión de sesiones también implica la asociación de datos con cada sesión. El API Servlet especifica la interfaz javax . servlet . HttpSession para proporcionar esta abstracción.

--Contexto de servlet La noción de contexto de servlet está íntimamente asociada a la noción de aplicación Web. La interfaz javax.servlet.ServletContext permite a los servlets de una aplicación compartir datos. También ofrece métodos con los que los servlets pueden acceder al contenedor Web host. Utilizando el objeto ServletContext, un servlet puede registrar eventos, obtener referencias URL a los recursos y fijar y almacenar atributos a los que puedan acceder otros servlets del contexto.

--Colaboración de servlet El API Servlet también proporciona una interfaz (RequestDispatcher) con la que un servlet puede invocar otro servlet, una JSP o incluso un recurso estático como una página HTML. Este mecanismo le ayuda a controlar el flujo de lógica de múltiples servlets y páginas JSP programáticamente.

--Filtrado El API Servlet tiene un mecanismo con el que podemos introducir código (llamado filtro) para participar en el proceso solicitud/respuesta del contenedor. Los filtros son una nueva característica del API Java Servlet. Generalmente, no crean solicitudes/respuestas; en su lugar, modifican o adaptan solicitudes y respuestas desde y hacia un recurso Web, que puede tener contenido estático o dinámico.

--Otros API varios En esta categoría, encontrará dos clases: http.Cookie, que puede utilizar para crear cookies y http.HttpUtils, que proporciona métodos ayudantes varios. Encontrará algunos ejemplos de estas clases en el siguiente capítulo.

Implementación de servlet

Comencemos examinando de cerca las clases e interfaces utilizadas para implementar servlets.

La interfaz servlet

```
public interface Servlet
```

Esta interfaz especifica el contrato entre el contenedor Web y un servlet. En el paradigma orientado al objeto, un objeto puede comunicar con otro siempre que el primer objeto pueda referenciar al segundo objeto con una interfaz conocida (no es necesario que conozca el nombre de la verdadera clase de implementación). En el caso del API Servlet, javax.servlet.Servlet es la interfaz que los contenedores utilizan para referenciar a servlets. ?

La configuración, se ve en el apartado

[11.7. Opciones de configuración del archivo web.xml](#)

Última modificación: lunes, 13 de marzo de 2023, 10:06